

Vsock api 文档 v2.0

GRTNBL20241118

v2.0 2024/12/07

公开 或 内部使用



目录

变更履历表	3
一. VSOCK 介绍	4
二. 使用方法	9
1. VSOCK 使用方法	9
2. VSOCKX 使用方法	11

变更履历表

版本	日期	修订人	修订内容
v0.1			init version
v1.0	2024/11/18	杜光辉	release version
v1.1	2024/11/30	杜光辉	添加 vsockx 使用方法
v2.0	2024/12/09	杜光辉	请 owner review 后修订部分内容

一. VSOCK 介绍

Vsocks 是一种在 VM 与其主机操作系统之间直接提供套接字通信（流或数据报）的方法。主机和每个 VM 都有一个 32 位的 CID（上下文标识符），并且可以连接或绑定到一个 32 位的端口号。< 1024 的端口是特权端口。

特殊值

有几个特殊地址：

VMADDR_CID_ANY(-1U)表示任何要绑定的地址；

VMADDR_CID_HYPERVISOR(0)保留用于管理程序中内置的服务；

VMADDR_CID_LOCAL(1)是本地通信(回送)的众所周知的地址；

VMADDR_CID_HOST(2)是主机的众所周知的地址。

特殊常数 **VMADDR_PORT_ANY**(-1U)表示要绑定的任何端口号。

VMADDR_CID_LOCAL(1)将数据包定向到生成数据包的同一主机。这对于在单个主机上测试应用程序和调试很有用。

可以将通过 **IOCTL_VM_SOCKETS_GET_LOCAL_CID** 获得的本地 CID 用于相同的目的，但是最好使用 **VMADDR_CID_LOCAL**。

名称	值	描述
VMADDR_PORT_ANY	-1	绑定到随机可用端口
VMADDR_CID_ANY	-1	绑定到 CID（只在虚拟机内生效）
VMADDR_CID_HYPERVISOR	0	管理程序的 CID

VMADDR_CID_RESERVED	1	预定的 CID
VMADDR_CID_HOST	2	主机 CID

struct sockaddr_vm

```
C++
struct sockaddr_vm {
    __kernel_sa_family_t svm_family;
    unsigned short svm_reserved1;
    unsigned int svm_port;
    unsigned int svm_cid;
    unsigned char svm_zero[sizeof(struct sockaddr) - sizeof(sa_family_t) -
sizeof(unsigned short) - sizeof(unsigned int) - sizeof(unsigned int)];
};
```

- svm_family 始终设置为 AF_VSOCK。 svm_reserved1 始终设置为 0。 svm_port 包含主机字节顺序的端口号。低于 1024 的端口号称为特权端口。只有具有 CAP_NET_BIND_SERVICE 功能的进程才能将(2)绑定到这些端口号。 svm_zero 必须为零。

ioctl(socket, IOCTL_VM_SOCKETS_GET_LOCAL_CID, &cid);

其中 IOCTL_VM_SOCKETS_GET_LOCAL_CID 获取本地计算机的 CID。该参数是一个指向无符号 int 的指针。

绑定时考虑使用 VMADDR_CID_ANY，而不是使用 IOCTL_VM_SOCKETS_GET_LOCAL_CID 获取本地 CID。

int socket (int af, int type, int protocol)

- af 参数：af 为协议族，地址族 (Address Family)，也就是 IP 地址类型，常用的有 AF_INET：IPv4 协议

- AF_INET6：IPv6 协议
- AF_LOCAL：UNIX 域协议
- AF_ROUTE：路由套接字
- AF_KEY：密钥套接字
- type 参数：types 指明通信字节流类型，其取值如：
 - SOCK_STREAM：流式套接字（TCP 方式）
 - SOCK_DGRAM：数据包套接字（UDP 方式）
 - SOCK_RAW：原始套接字
- protocol 参数：protocol 表示传输协议，常用的有 IPPROTO_TCP 和 IPPROTO_UDP，分别表示 TCP 传输协议和 UDP 传输协议。通常设置为 0。

成功时返回文件描述符，失败时返回-1，并设置 errno。

```
int bind (int sock, struct sockaddr *addr, socklen_t  
addrlen) ;
```

- sock 参数：sock 为 socket 描述符。
- addr ：绑定的地址
- addrlen：地址长度

成功时返回 0，失败时返回-1，并设置 errno；

```
int connect (int sock, struct sockaddr *serv_addr,  
socklen_t addrlen) ;
```

- sock 参数：sock 为 socket 描述符。
- addr ：绑定的地址
- addrlen：地址长度

成功时返回 0，失败时返回-1，并设置 errno；

```
int listen(int sock,int backlog);
```

- sock 参数：sock 为需要进入监听状态的套接字。
- backlog 参数：backlog 为请求队列的最大长度。

成功时返回 0，失败时返回-1，并设置 errno；

```
int accept (int sock, struct sockaddr *addr,
socklen_t *addrlen)
```

- sock 参数：sock 为服务器端套接字
- addr 参数：addr 为 sockaddr_in 结构体变量。
- addrlen 参数：addrlen 为参数 addr 的长度，可由 sizeof() 求得。

返回值：一个新的套接字，用于与客户端通信。

```
int close(int fd);
```

- -fd :要关闭的文件描述符

数据的接收和发送

```
ssize_t read(int fd, void* buf, size_t nbytes);
```

- fd：将要从该 fd 上读取数据；
- buf：保存所读取数据的缓冲区；
- nbytes：能够读取的最大字节数；

size_t 与 ssize_t 是通过 typedef 声明定义的元数据类型，主要是兼容不同位数的操作系统数据类型的差异（如 int32、int64），为了与用户自己定义的数据类型区别开，以_t 结尾。

--成功时返回字节数，遇到文件结尾返回 0，失败时返回-1；

```
ssize_t write(int fd, const void* buf, size_t nbytes);
```

- fd：数据想要写入的 sockfd；
- buf：保存将要写入数据的缓冲区；
- nbytes：将要写入的字节数

与 write()相对应的是 read()函数，用来读取数据

--成功时返回写入的字节数，失败时返回-1；

错误说明

- EACCES

没有 CAP_NET_BIND_SERVICE 功能，无法绑定到特权端口。

- EADDRINUSE

无法绑定到已使用的端口。

- EADDRNOTAVAIL

无法找到用于绑定的空闲端口或无法绑定到非本地 CID。

- EINVAL

无效的参数。这包括：尝试绑定已经绑定的套接字，提供无效的 struct sockaddr_vm，以及其他输入验证错误。

- ENOPROTOPT

setsockopt(2)或 getsockopt(2)中的套接字选项无效。

- ENOTCONN

无法在未连接的套接字上执行操作。

- EOPNOTSUPP

不支持该操作。这包括：未针对 send(2)系列的系统调用实现的 MSG_OOB 标志和针对 recv(2)系列的系统调用的 MSG_PEEK。

- EPROTONOSUPPORT

无效的套接字协议号。协议应始终为 0。

- ESOCKTNOSUPPORT

套接字(2)中不支持的套接字类型。仅 SOCK_STREAM 和 SOCK_DGRAM 有效。

二. 使用方法

1. VSOCK 使用方法

server 端通讯案例

```
C++
int main()
{
    int s = socket(AF_VSOCK, SOCK_STREAM, 0);
    const struct sockaddr_vm addr = {
        .svm_family = AF_VSOCK,
        .svm_prot = 9999,
        .svm_cid = VMADDR_CID_HOST,
    }
    bind(s, (const sockaddr *)&addr, sizeof(struct sockaddr_vm));

    listen(s, 0);

    struct sockaddr_vm peer_addr;
    socklen_t peer_addr_size = sizeof(struct sockaddr_vm);
    int peer_fd = accept(s, (sockaddr *)&peer_addr, &peer_addr_size);

    char buf[54];
    ssize_t msg_len;
    while ((msg_len = recv(peer_fd, &buf, 64, 0)) > 0){
        printf("Received %zd bytes: %.*s\n", msg_len, msg_len, buf);
    }
    return 0;
}
```

client 端通讯案例

```
C++  
  
int main()  
{  
    int s = socket(AF_VSOCK, SOCK_STREAM, 0);  
  
    struct sockaddr_vm ddr;  
    memset(&addr, 0, sizeof(struct sockaddr_vm));  
    addr.svm_family = AF_VSOCK;  
    addr.svm_prot = 9999;  
    addr.svm_cid = VMADDR_CID_HOST;  
  
    int rc = connect(s, &addr, sizeof(struct sockaddr_vm));  
    if(rc < 0){  
        printf("rc = %d %s\n", rc, strerror(errno));  
        return rv;  
    }  
    send(s "Hello,world!", 13, 0);  
  
    close(s);  
  
    return 0;  
}
```

2. VSOCKX 使用方法

vsockx 是谦川基于 vsock 开发的 UOS 之间 socket 通信的方式，区别在于实现方式和用法，客户只需要关心用法即可

对于原生的 vsock

tbox CID:4

android CID:3

SOS CID:2 //VMADDR_CID_HOST

对于 UOS 间 vsock, 即 vsockx

tbox CID:4

android CID: 2 //VMADDR_CID_HOST

server 端通讯案例

C++

```
#define AF_VSOCKX AF_DECnet
```

```
int main()
```

```
{
```

```
    int s = socket(AF_VSOCKX, SOCK_STREAM, 0);
```

```
    const struct sockaddr_vm addr = {
```

```
        .svm_family = AF_VSOCK,
```

```
        .svm_prot = 9999,
```

```
        .svm_cid = VMADDR_CID_HOST,
```

```
    }
```

```
    bind(s, (const sockaddr *)&addr, sizeof(struct sockaddr_vm));
```

```
    listen(s, 0);
```

```

struct sockaddr_vm peer_addr;

socklen_t peer_addr_size = sizeof(struct sockaddr_vm);

int peer_fd = accept(s, (sockaddr *)&peer_addr, &peer_addr_size);

char buf[54];

ssize_t msg_len;

while ((msg_len = recv(peer_fd, &buf, 64, 0)) > 0){
    printf("Received %zd bytes: %.*s\n", msg_len, msg_len, buf);
}

return 0;
}

```

client 端通讯案例

```

C++

#define AF_VSOCKX AF_DECnet

int main()
{
    int s = socket(AF_VSOCKX, SOCK_STREAM, 0);

    struct sockaddr_vm ddr;
    memset(&addr, 0, sizeof(struct sockaddr_vm));
    addr.svm_family = AF_VSOCK;
    addr.svm_prot = 9999;
    addr.svm_cid = VMADDR_CID_HOST;

    int rc = connect(s, &addr, sizeof(struct sockaddr_vm));

    if(rc < 0){

```

```
    printf("rc = %d %s\n", rc, strerror(errno));

    return rv;
}

send(s "Hello,world!", 13, 0);

close(s);

return 0;
}
```